

SOEN 201 – Object Oriented Analysis & Design

Lecturer: Catherine | Kisii University | Expanded Notes with Diagrams

1. Class and Object – The Foundation

Class

A class is a blueprint or template that describes the attributes (data) and methods (behaviour) that its objects will have. It does not occupy memory for the actual data until an object is created from it.

Object

An object is an **instance** of a class. It is a concrete thing that has the attributes and behaviour defined by its class. Anything that has attributes (properties) and behaviour (actions) can be modelled as an object.

Example – Car

The **class** Car describes what every car has and can do. Each real car is an **object** (instance) of that class.

Attributes (data): Colour, Shape/Size, Number of doors, Number of wheels, Model, Brand

Methods (actions): start(), stop(), accelerate()

When we create Car `myCar = new Car()`; we get one object. We can create many objects from the same class, each with different attribute values.

2. From Procedural to Object-Oriented Thinking

Procedural Programming – Focuses on procedures (functions) that operate on data. Data and functions are often separate.

Modular Programming – Breaks a program into smaller independent modules, but data and behaviour are still not tightly bound.

Object-Oriented Programming – Combines data (attributes) and operations (methods) into a single unit called an object. This is data abstraction.

Object Orientation is a way of viewing the world in terms of interrelated objects. We ask “What objects exist and how do they interact?” instead of only “What steps should the program take?”

3. Core Principles of Object Orientation

3.1 Abstraction

Hiding complexity by providing a simple interface. We focus on **what** an object does, not on **how** it does it. Example: Driving a car — you use the steering wheel and pedals; you do not need to know the internal engine details.

3.2 Encapsulation

Bundling data and methods into one unit and controlling access (usually private attributes + public methods). Example: A Student class keeps marks private; other code must call `setMarks()` which can validate the value.

3.3 Polymorphism

Different objects can respond to the same message in different ways. Example: `draw()` called on a Circle, Rectangle or Triangle — each implements it differently.

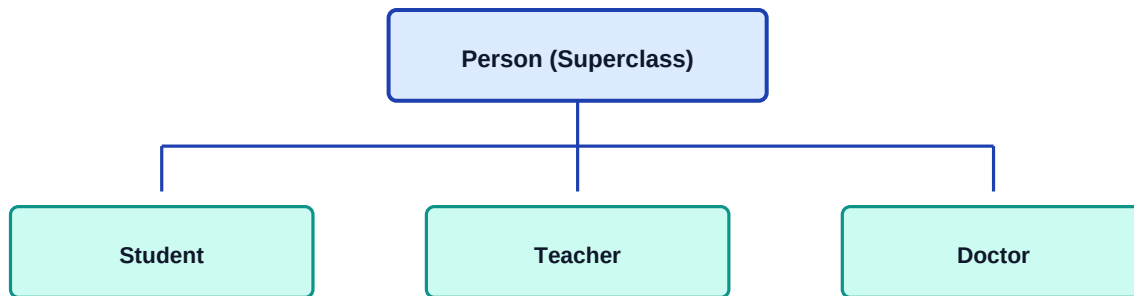
3.4 Modularity

Breaking a system into smaller, independent units (classes) each with a clear responsibility. Makes the system easier to understand and maintain.

3.5 Inheritance / Hierarchy

A class can inherit attributes and methods from another class. The class that inherits is the **subclass** (child). The class being inherited from is the **superclass** (parent). Inheritance creates an “is-a” relationship.

Visual example – Person hierarchy



Person is the superclass. Student, Teacher and Doctor are subclasses (they inherit common features and add their own).

Student adds: registration number, course, year of study.

Teacher adds: employee number, subjects, salary.

Doctor adds: licence number, specialisation, hospital.

4. Relationships Between Classes

Objects interact. In design we show these interactions as relationships. Your lecturer represents them with specific diamond symbols — these are important for the CAT.

4.1 Association

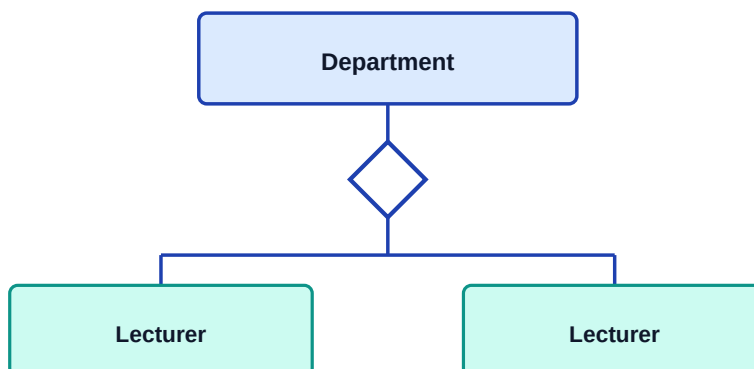
A general relationship between two classes at the same level. One object uses or knows about another. No ownership is implied. Drawn as a simple line (no diamond).



Association — simple “uses / knows about” link (no diamond)

4.2 Aggregation

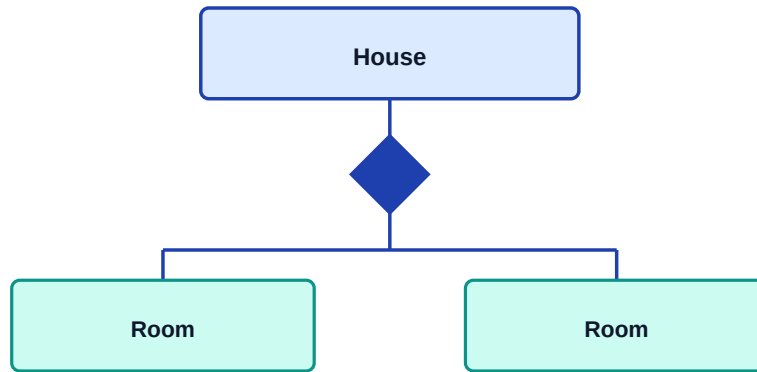
A “has-a” relationship where the **part can exist independently** of the whole. In diagrams it is shown with a **hollow (clear) diamond** next to the whole.



Example: A Department has Lecturers. If the Department is closed, the Lecturers still exist and can move elsewhere.

4.3 Composition

A stronger “has-a” relationship. The **part cannot exist without the whole**. If the whole is destroyed, the parts are also destroyed. In diagrams it is shown with a **filled (coloured) diamond** next to the whole — exactly as your lecturer drew it.



Example: A House has Rooms. If the House is demolished, the Rooms no longer exist.

Quick memory aid

- Association → simple line → “uses / knows about”
- Aggregation → hollow diamond → “has-a” (part can live alone)
- Composition → filled diamond → “has-a” (part dies with the whole)

5. Quick Revision Checklist

- Class = blueprint; Object = concrete instance of that blueprint.
- Attributes = data; Methods = behaviour.
- Abstraction hides complexity; Encapsulation protects data.
- Polymorphism = same message, different behaviour.
- Inheritance = “is-a” hierarchy (subclass inherits from superclass).
- Hollow diamond = Aggregation; Filled diamond = Composition.

These notes expand the class material from SOEN 201 (Catherine). The diagrams follow the exact diamond style your lecturer used. Practise drawing them yourself before the CAT.